

Dokumentasi Branch `dev-bayu` — JaGOOD ColdChain

Smart Route Planner: layanan AI rekomendasi rute untuk logistik rantai dingin

Branch	<code>dev-bayu</code> (2 commit di atas <code>main</code>)
Cakupan	89 file, +27.910 baris
Commit	<code>d0d9c72</code> → <code>d89c681</code>
Status	Modul Smart Route Planner berfungsi penuh <i>end-to-end</i>
Dokumen per	12 Agustus 2026

1. Ringkasan

JaGOOD ColdChain adalah solusi berbasis AI untuk membuat pengiriman pangan rantai dingin lebih aman dan terpantau. Produk seperti hasil laut, daging, susu, dan sayuran segar membutuhkan suhu stabil sepanjang perjalanan; perubahan rute, keterlambatan, atau gangguan cuaca dapat menurunkan mutu dan memperpendek umur simpan.

Branch `dev-bayu` membangun **satu dari empat fitur** yang direncanakan — **Smart Route Planner** — secara lengkap dari model machine learning hingga antarmuka peta interaktif.

Status keempat fitur produk

Fitur	Status di <code>dev-bayu</code>
Smart Route Planner	☒ Berfungsi penuh (FastAPI + XGBoost + Next.js)
AI Scenario Simulator	☐ Belum diimplementasikan (folder + README saja)
Transportation Monitoring	☐ Belum diimplementasikan
AI Explain (sebagai service terpisah)	☐ Belum — namun explainability SHAP sudah tertanam di route-planner

Layanan pendukung `weather/` , `notification/` , `authentication/` , dan `backend/` (API gateway) juga masih berupa placeholder.

2. Struktur Repositori

frontend/	Next.js 16 + React 19 – dashboard peta interaktif
backend/	placeholder API gateway (belum diimplementasikan)
services/	
ai/	
route-planner/	★ satu-satunya service yang berjalan
app/	
core/	config (pydantic-settings) + cache Postgres
data/	komoditas, pelabuhan, GeoJSON BMKG, data sintetis
ml/	feature pipeline (dipakai training & serving)
models/	model_pipeline.pkl + model_metadata.json
routers/	endpoint FastAPI
schemas/	kontrak request/response Pydantic
services/	routing, enrichment, ranking, SHAP, simulasi suhu
training/	generator data sintetis + training + evaluasi
scripts/	validasi skenario ekstrem
scenario-simulator/	placeholder
monitoring/	placeholder
ai-explain/	placeholder
weather/ notification/ authentication/	placeholder
datasets/	kosong (data route-planner menyatu dengan service-nya)
docs/	dokumentasi proyek
infrastructure/	docker-compose (Postgres + backend + frontend)

3. Arsitektur Alur Permintaan

Satu panggilan `POST /predict-route` melewati empat tahap berurutan:

1. `route_generator.py` → hasilkan 2-4 kandidat rute
 - darat via `OpenRouteService` (varian: `recommended` / `shortest` / `hindari tol`)
 - kombinasi darat-laut-darat via `port_selector` + `searoute-py`↓
2. `enrichment_service.py` → lengkapi tiap kandidat dengan:
 - data komoditas (suhu ideal, umur simpan, toleransi)
 - baseline historis per koridor
 - cuaca & gelombang BMKG (live)
 - simulasi suhu kargo (khusus mode pasif)↓
3. `ranking_service.py` → prediksi risiko XGBoost, terapkan override `tau_high`, tandai kondisi ekstrem, lalu urutkan
- ↓
4. `explanation_service.py` → SHAP per rute → kalimat penjelas Bahasa Indonesia

Seluruh panggilan API eksternal dijalankan konkuren (`ThreadPoolExecutor` untuk ORS yang *blocking*, `asyncio.gather` untuk BMKG/Open-Meteo) demi menekan latensi.

4. AI 1 — Cold Chain Risk Prediction

Memprediksi tingkat risiko penurunan mutu produk selama distribusi.

4.1 Model

Aspek	Nilai
Algoritma	<code>XGBClassifier</code> (<code>multi:softprob</code>)
Kelas	<code>Low</code> / <code>Medium</code> / <code>High</code>
Hiperparameter	<code>max_depth=5</code> , <code>n_estimators=300</code> , <code>learning_rate=0.08</code> , <code>subsample=0.9</code> , <code>colsample_bytree=0.9</code>
Jumlah fitur	18 fitur input + 3 fitur interaksi
Ukuran artefak	1,63 MB (<code>model_pipeline.pkl</code>)

XGBoost dipilih karena unggul pada data tabular, cepat dilatih, bekerja baik pada dataset yang belum besar, dan interpretabilitasnya tinggi — cocok untuk MVP hackathon.

4.2 Fitur input

Kategorikal (5): `commodity_type` , `transport_mode` , `weather_condition` , `wave_category` , `cold_chain_equipment`

Numerik (13): `commodity_temp_ideal_c` , `commodity_shelf_life_hours` , `commodity_delay_tolerance_hours` , `distance_km` , `estimated_duration_hours` , `wave_height_m` , `wind_speed_kmh` , `port_status_flag` , `historical_delay_avg_hours` , `historical_damage_rate` , `departure_hour` , `port_ambient_temp_c` , `max_cargo_temp_excess_c`

Interaksi (3): hasil rekayasa fitur yang mengalikan sensitivitas suhu komoditas dengan tekanan eksternal — `wave_temp_interaction` , `port_temp_interaction` , `cold_chain_temp_interaction` .

Pemetaan terhadap spesifikasi produk:

Spesifikasi	Implementasi
weather	<code>weather_condition</code> , <code>wave_category</code> , <code>wave_height_m</code> , <code>wind_speed_kmh</code> (BMKG live)
ETA	<code>estimated_duration_hours</code> + <code>departure_hour</code>
commodity	4 atribut komoditas dari basis data 10 komoditas
transport mode	<code>transport_mode</code> (darat / laut / kombinasi)
distance	<code>distance_km</code>
delay history	<code>historical_delay_avg_hours</code> , <code>historical_damage_rate</code>

4.3 Strategi anti *false negative*

Kesalahan paling mahal pada rantai dingin adalah gagal mendeteksi risiko `High` . Tiga mekanisme diterapkan:

- 1. **Bobot sampel** — kelas `High` diberi bobot $1,75\times$ di atas *balanced weighting*.
- 2. **Ambang `tau_high`** — di-*tuning* pada slice validasi (bukan test set) untuk menjamin `recall_High` $\geq 0,80$; prediksi di-*override* menjadi `High` bila `P(High)` $\geq$ `tau_high` .
- 3. **Split kronologis** — data diurutkan waktu, bukan diacak, untuk meniru kondisi deployment nyata.

4.4 Hasil evaluasi terverifikasi

Dijalankan ulang penuh (generate → train → evaluate) pada 12 Agustus 2026.

Dataset : 14.353 pengiriman sintetis, 200 koridor
Split : train 9.759 / validasi 1.723 / test 2.871 (kronologis)
Periode : train Agu 2024–Des 2025 · test Mar–Jul 2026
tau_high: 0,99971

	precision	recall	f1-score	support
Low	0,974	0,973	0,973	1441
Medium	0,953	0,949	0,951	894
High	0,985	0,994	0,990	536
accuracy			0,969	2871

Confusion matrix (baris = aktual, kolom = prediksi)

	Low	Medium	High
Low	1402	39	0
Medium	38	848	8
High	0	3	533

recall_High = 0,994 (target PRD $\geq$ 0,80) → LULUS

Interpretasi jujur: tidak ada satu pun kebingungan `Low` ↔ `High` di kedua arah; seluruh kesalahan terjadi antar kelas bersebelahan. Ini bukan bukti model unggul, melainkan indikasi bahwa fungsi label sintetis pada dasarnya adalah ambang monoton yang berhasil direkayasa-balik oleh model. Angka ini mengukur *learnability* data sintetis, **bukan** akurasi terhadap kerusakan pangan sungguhan.

Konsekuensi lain: karena model dasarnya sudah nyaris sempurna, `tau_high` ter-*tuning* ke 0,9997 sehingga mekanisme *override* praktis **tidak pernah aktif**. Desainnya benar, tetapi saat ini dorman dan belum teruji — mekanisme ini baru akan berperan ketika dilatih dengan data nyata yang jauh lebih berderau.

4.5 Dua angka keluaran yang berbeda

API mengembalikan dua metrik yang sering tertukar:

Field	Rumus	Makna
<code>risk_probability</code>	$P(\text{Medium}) \times 0,5 + P(\text{High}) \times 1,0$	Skor keparahan 0–1 — dipakai untuk pengurutan
<code>confidence_score</code>	$\max(\text{proba})$	Keyakinan model pada kelas yang dipilihnya

Antarmuka sengaja menamainya "**Skor risiko**", bukan "Probabilitas". `risk_probability` adalah indeks keparahan tertimbang, **bukan** peluang barang rusak — hindari menyebutnya "probabilitas" saat presentasi.

5. AI 2 — Smart Route Recommendation

Setelah risiko diketahui, sistem memilih jalur terbaik. Berbeda dengan peta umum yang mengejar rute tercepat, JaGOOD mengutamakan **risiko kerusakan produk terendah**.

5.1 Pendekatan

Untuk MVP dipilih *ranking engine* berbasis skor prediksi risiko — bukan *learning-to-rank*. Dengan hanya 2–4 kandidat per permintaan, LTR akan berlebihan dan lebih sulit dijelaskan kepada juri.

5.2 Parameter `ranking_preference`

Nilai	Kunci pengurutan	Perilaku
<code>risiko</code> (default)	<code>["risk_probability", "estimated_duration_hours"]</code>	Risiko terendah menang; durasi hanya pemecah seri
<code>kecepatan</code>	<code>["estimated_duration_hours"]</code>	Murni tercepat; risiko tetap dihitung & ditampilkan

Nilai tak dikenal **jatuh ke** `risiko`, bukan ke kecepatan — karena diam-diam mengurutkan kiriman rantai dingin berdasarkan kecepatan adalah mode kegagalan yang lebih berbahaya.

5.3 Bukti perilaku

Uji dengan dua kandidat berlawanan sifat:

Kandidat A (FAST-risky) : 20,0 jam • gelombang "Sangat Tinggi" • badai • pelabuhan berisiko

Kandidat B (SLOW-safe) : 34,0 jam • darat • cuaca cerah

`ranking_preference=risiko` → direkomendasikan: SLOW-safe (Low, $p=0,000$)

`ranking_preference=kecepatan` → direkomendasikan: FAST-risky (High, $p=1,000$)

Kandidat identik, rekomendasi berlawanan — inilah pembeda produk yang dapat didemonstrasikan langsung.

Pada data nyata Jakarta → Makassar, mode `risiko` memilih rute yang **1,25 jam lebih lambat** karena risikonya lebih rendah.

5.4 Keluaran

Output spesifikasi	Field API
Recommended Route	<code>recommended_route</code> (+ <code>alternative_routes</code>)
Estimated Risk	<code>risk_level</code> + <code>risk_probability</code>
Estimated Arrival	<code>estimated_arrival</code>
Confidence Score	<code>confidence_score</code>

6. Explainability (SHAP)

`explanation_service.py` menghitung SHAP `TreeExplainer` pada ruang *margin* (pra-softmax), lalu menggabungkan kontribusi kelas Medium dan High dengan bobot 0,5/1,0 — bobot yang sama dengan `risk_probability` . Tiga faktor teratas dirender menjadi kalimat Bahasa Indonesia yang natural.

Contoh keluaran nyata:

Risiko Sedang lebih terkendali karena status pelabuhan normal, namun riwayat tingkat kerusakan rute serupa (33%) tetap menaikannya.

- ↓ Status pelabuhan normal
- ↑ Riwayat tingkat kerusakan rute serupa (33%)
- ↓ Kombinasi gelombang tinggi dengan komoditas sensitif suhu

Batasan: kontribusi di ruang margin tidak menjumlah linear ke metrik ruang probabilitas. Ini adalah pendekatan praktis standar untuk memeringkat *faktor mana* yang mendorong prediksi model pohon multi-kelas dan ke arah mana — perlakukan sebagai penjelasan kualitatif, bukan dekomposisi probabilitas yang eksak.

7. Simulasi Suhu Kargo (Q10)

Berlaku hanya saat `cold_chain_equipment = "pasif"` (tanpa pendingin aktif). Untuk `reefer` (default), kargo diasumsikan tetap pada suhu ideal — tanpa panggilan API tambahan.

Model fisika: suhu kargo meluruh secara eksponensial menuju suhu udara sekitar nyata (Open-Meteo, gratis tanpa API key) yang disampel di 6 titik sepanjang rute. Konsumsi umur simpan dipercepat mengikuti aturan **Q10 = 2,5** setiap kali suhu melampaui titik ideal komoditas.

Kualitas insulasi	Konstanta transfer panas (per jam)
baik (cooler box tebal)	0,08
sedang (styrofoam standar)	0,15
buruk (kardus/insulasi tipis)	0,35

Asumsi yang perlu diketahui: suhu awal kargo dianggap sudah pada titik ideal (pra-pendinginan benar), kualitas insulasi dipetakan ke konstanta tetap (bukan hasil pengukuran), dan **cadangan es/phase-change tidak dimodelkan** — pendingin pasif nyata dengan ice pack akan bertahan lebih lama daripada prediksi model ini.

8. Data Sintetis

Data historis dibangkitkan dua lapis:

- **Lapis A** (`synthetic_corridors.py`) — 200 koridor (darat 70, laut 80, kombinasi 50) dengan baseline keterlambatan dan tingkat kerusakan yang stabil.
- **Lapis B** (`generate_synthetic_data.py`) — 14.353 pengiriman bertanggal di atas koridor tersebut, dengan kondisi dinamis per pengiriman.

Derau disuntikkan pada **pemicu kontinu** (sampling gelombang gamma, derau keterlambatan, derau beta pada tingkat kerusakan), bukan dengan membalik label akhir — sehingga model menghadapi batas keputusan yang nyata, bukan tabel pencarian.

Distribusi kelas: Low 50% / Medium 30% / High 20%.

⚠ Angka ini dihasilkan ambang `low=62,1` dan `high=77,2` yang di-*hardcode* — jelas hasil pencocokan kuantil terbalik, bukan turunan batasan domain rantai dingin. Distribusi risiko dunia nyata hampir pasti tidak sebulat ini.

Reprodusibilitas: dengan `RNG_SEED = 7`, seluruh pipeline regenerasi **byte-identik**. Hanya `model_pipeline.pkl` yang berbeda tipis antar-latihan (non-determinisme *threading* XGBoost).

9. Referensi API

Base URL pengembangan: `http://localhost:8000` · Swagger: `/docs` · ReDoc: `/redoc`

GET /health

```
{"status": "ok"}
```


GET /commodities

Mengembalikan 10 komoditas beserta rentang suhu ideal, umur simpan, toleransi keterlambatan, dan tingkat sensitivitas suhu.

POST /predict-route

Request

Field	Tipe	Default	Keterangan
origin / destination	{lat, lon}	wajib	Titik asal dan tujuan
commodity_type	string	wajib	Harus ada di /commodities
departure_time	datetime ISO	wajib	Waktu keberangkatan
transport_mode_preference	darat laut kombinasi semua	semua	Filter moda
cold_chain_equipment	reefer pasif	reefer	Memicu simulasi suhu bila pasif
insulation_quality	baik sedang buruk	sedang	Hanya dipakai saat pasif
ranking_preference	risiko kecepatan	risiko	Kunci pengurutan rute
shipment_id	string	auto	Dibuat otomatis bila kosong

Response — recommended_route + alternative_routes , masing-masing memuat: identitas rute, distance_km , estimated_duration_hours , estimated_arrival , risk_level , risk_probability , confidence_score , trigger_reason , data_quality , kondisi cuaca/gelombang BMKG, port_ambient_temp_c , baseline historis, max_cargo_temp_excess_c , cargo_temp_profile , geometry (pasangan [lat, lon]), risk_hotspot , port_pair , serta risk_explanation_summary dan risk_explanation_factors .

Kode status: 200 sukses · 404 komoditas tidak dikenal · 422 tidak ada kandidat rute.

10. Frontend

Next.js 16 (App Router, Turbopack) + React 19 + Tailwind CSS 4, peta interaktif via react-leaflet .

Komponen	Fungsi
RouteMap	Peta Leaflet: garis rute berwarna sesuai risiko, penanda A/B yang dapat diseret, ikon pelabuhan, titik risiko
CargoTempChart	Grafik profil suhu kargo hasil simulasi Q10
RiskBadge	Lencana Rendah / Sedang / Tinggi
RiskExplanation	Render penjelasan SHAP beserta arah pengaruh tiap faktor
ParameterLegend	Keterangan seluruh parameter dan ikon peta

Formulir menyediakan pilihan asal/tujuan (13 kota preset atau titik kustom dari peta), komoditas, waktu keberangkatan, preferensi moda, **urutan rute**, peralatan rantai dingin, dan kualitas insulasi.

11. Panduan Menjalankan

Prasyarat

- Python 3.12
- Node.js 20+
- PostgreSQL yang dapat dijangkau (hanya untuk cache BMKG/Open-Meteo)
- API key OpenRouteService (gratis di openrouteservice.org)

Backend

```
cd services/ai/route-planner
python -m venv .venv
.venv/Scripts/pip install -r requirements.txt      # .venv/bin/pip di macOS/Linux

cp .env.example .env                             # isi ORS_API_KEY & DATABASE_URL

python -m training.synthetic_corridors
python -m training.generate_synthetic_data
python -m training.train_model
python -m training.evaluate_model

python -m uvicorn app.main:app --reload
```

Frontend

```
cd frontend
npm ci
npm run dev          # http://localhost:3000
```

Konfigurasi `.env`

```
ORS_API_KEY=<kunci OpenRouteService Anda>
BMKG_BASE_URL=https://peta-maritim.bmkg.go.id/public_api
DATABASE_URL=postgresql+psycopg2://user:password@host:5432/database
BMKG_CACHE_TTL_SECONDS=10800
```

`.env` sudah tercantum di `.gitignore` — jangan pernah meng-commit kredensial.

Validasi skenario ekstrem

```
python -m scripts.validate_scenarios
```

Menguji beberapa kombinasi komoditas/rute/musim plus satu kasus cuaca ekstrem yang dipaksakan, memvalidasi bahwa kondisi ekstrem **menurunkan peringkat** kandidat dan mengisi `trigger_reason` — bukan memblokirnya secara keras.

Catatan lingkungan Windows

Docker Desktop dan WSL2 memerlukan virtualisasi perangkat keras yang aktif di BIOS/UEFI (AMD-V/SVM atau Intel VT-x). Bila `systeminfo` menampilkan `Virtualization Enabled In Firmware: No`, aktifkan lebih dulu melalui menu firmware. Sebagai alternatif, PostgreSQL native atau instans remote bekerja tanpa virtualisasi sama sekali.

12. Ketergantungan Eksternal

Sumber	Kegunaan	Kunci?	Cache
OpenRouteService	Rute jalan raya (profil <code>driving-hgv</code>)	Ya	✗ Tidak
BMKG Maritim	Prakiraan gelombang, cuaca, suhu pelabuhan	Tidak	✓ Postgres
Open-Meteo	Suhu udara sekitar (mode pasif)	Tidak	✓ Postgres
searoute-py	Jalur laut untuk estimasi jarak & visualisasi	—	Lokal
PostgreSQL	Penyimpanan cache	—	—

13. Batasan yang Diketahui

Bagian ini sengaja ditulis terbuka untuk juri dan peninjau.

1. **Data historis sepenuhnya sintetis.** Dihasilkan model berbasis aturan dengan derau, belum pernah divalidasi terhadap catatan pengiriman nyata. `recall_High = 0,994` mencerminkan seberapa mudah fungsi label sintetis dipelajari, bukan performa dunia nyata.
2. `port_status_flag` adalah **proksi** yang diturunkan dari kategori gelombang BMKG, bukan status operasional pelabuhan resmi (tidak tersedia API Inaportnet publik).
3. `port_ambient_temp_c` adalah **proksi** tekanan pada unit pendingin selama menunggu di pelabuhan, **bukan** suhu kargo sesungguhnya. Kontribusinya ke model sengaja dibuat kecil (*feature importance* $\approx 0,001-0,0014$).
4. `searoute-py` bukan **alat navigasi maritim** — hanya untuk estimasi jarak/durasi dan visualisasi.
5. **Instans publik OpenRouteService memiliki celah data jalan di Indonesia** (teramati: perjalanan 13 km di Jakarta terpetakan menjadi memutar >1.500 km). Setiap segmen diperiksa terhadap jarak garis lurus dan diganti estimasi haversine bila tidak masuk akal; kandidat terdampak ditandai `data_quality: "estimated"` .
6. **SHAP dihitung di ruang margin** — penjelasan kualitatif, bukan dekomposisi probabilitas eksak (lihat §6).
7. **ORS tidak di-cache.** README service menyatakan respons ORS ikut di-cache, namun `get_cached` hanya dipakai di `weather_service.py` dan `temperature_service.py` . `route_generator.py` memanggil ORS tanpa cache pada setiap permintaan.
8. **Latensi di atas target.** Permintaan multimoda memakan **~14 detik** meski cache sudah hangat, jauh di atas target PRD <5 detik. Penyebab utama diduga tidak adanya cache ORS ditambah *round-trip* jaringan ke Postgres remote.

9. Pengurutan membedakan selisih risiko yang tak berarti. Teramati dua rute berbeda hanya $8,76 \times 10^{-6}$ pada `risk_probability`, dan itu cukup untuk merekomendasikan rute 1,25 jam lebih lambat. Disarankan menambah toleransi (misalnya pembulatan 3 desimal) agar risiko yang praktis setara jatuh ke pemecah seri durasi.
10. Ambang kelas di-*hardcode* (`62,1` / `77,2`) untuk menghasilkan distribusi 50/30/20 (lihat §8).

14. Riwayat Perubahan

d0d9c72 — Membangun service Smart Route Planner

Restrukturisasi monorepo, service FastAPI + XGBoost, pipeline data sintetis, integrasi BMKG, `docker-compose` Postgres. Pengurutan awal: `["risk_probability", "estimated_duration_hours"]`.

d89c681 — Peta interaktif, simulasi suhu, SHAP

Peta Leaflet dengan penanda yang dapat diseret, simulasi suhu kargo Q10 untuk mode pasif, explainability SHAP per rute, varian rute darat (tol/terpendek). Pengurutan diubah menjadi murni durasi.

Perubahan belum di-commit

10 file, +110/−15 baris:

Berkas	Perubahan
<code>app/services/ranking_service.py</code>	<code>RANKING_SORT_KEYS</code> + parameter <code>ranking_preference</code> , mengembalikan pengurutan berbasis risiko sebagai default
<code>app/schemas/route_schema.py</code>	Tipe <code>RankingPreference</code> , field <code>ranking_preference</code> dan <code>estimated_arrival</code>
<code>app/services/enrichment_service.py</code>	Perhitungan <code>estimated_arrival</code>
<code>app/routers/route_planner.py</code>	Meneruskan preferensi pengurutan
<code>app/core/db.py</code>	Pembuatan skema menjadi <i>lazy</i> — melepas ketergantungan script training pada Postgres
<code>frontend/src/app/page.tsx</code>	Dropdown urutan rute, kolom "Tiba", label mode aktif, perbaikan subtitle
<code>frontend/src/lib/types.ts</code>	Field <code>estimated_arrival</code>
<code>README.md</code> (service)	Dokumentasi <code>ranking_preference</code>
<code>app/models/*</code>	Artefak hasil latih ulang

Catatan `db.py`: sebelumnya `metadata.create_all(engine)` dijalankan saat *import*, sehingga script training — yang hanya membutuhkan konstanta dan fungsi simulasi suhu — tetap memerlukan Postgres aktif hanya

agar dapat di-*import*. Kini skema dibuat saat cache pertama kali digunakan.

15. Rekomendasi Prioritas

Prioritas	Item	Alasan
● Tinggi	Tambahkan cache ORS	Menurunkan latensi ~14 s yang melanggar target PRD
● Tinggi	Toleransi pada pengurutan risiko	Mencegah rekomendasi rute lebih lambat karena selisih 10^{-6}
● Sedang	Tangani kegagalan cache secara <i>graceful</i>	Cache adalah optimasi; Postgres mati seharusnya tidak mematikan seluruh permintaan
● Sedang	Aktifkan SSL untuk koneksi Postgres	Koneksi saat ini melintasi internet publik tanpa enkripsi
● Rendah	CORS yang dapat dikonfigurasi	<code>main.py</code> masih <i>hardcode</i> <code>http://localhost:3000</code>
● Rendah	Uji otomatis	Belum ada suite pengujian

Dokumen ini menjelaskan branch `dev-bayu` per 12 Agustus 2026. Seluruh metrik model diperoleh dari menjalankan ulang pipeline training dan evaluasi secara penuh, bukan dikutip dari dokumentasi sebelumnya.